Introduction

Every so often a LeetCode problem looks like one you've already solved, and that's exactly where it gets you. Climbing Stairs II (LeetCode 3693) borrows its skeleton from the classic "count the ways to climb stairs" question, but swaps counting for cost. Suddenly the instinct that worked before — take the biggest jump, cover more ground, finish faster — actively works against you.

Interviewers like this family of problems because it tests something more specific than "do you know dynamic programming." It tests whether you notice when a greedy shortcut *looks* correct but silently breaks under a cost function. That's a skill you use constantly in real engineering: caching strategies, routing decisions, resource allocation — anywhere "fewer steps" gets confused with "cheaper."

Learning this problem properly gives you two things: a reusable DP pattern (answer at step `j` depends only on a fixed, small window of previous steps), and a sharper intuition for when greedy reasoning quietly fails.

Problem Overview

Picture a staircase with steps numbered `0` through `n`. You start standing on step `0`, having spent nothing. From any step `i`, you're allowed to jump forward to `i + 1`, `i + 2`, or `i + 3` — but never further, and never past step `n`.

Every step `j` (except step `0`) has its own fixed cost, given in an array `cost`. When you jump and land on step `j`, you pay `cost[j]` plus the **square** of the distance you just jumped. So a 1-step jump adds `1`, a 2-step jump adds `4`, and a 3-step jump adds `9` — on top of whatever that landing step already costs.

Your job: find the minimum total cost to go from step `0` to step `n`.

The squared penalty is the whole story here. It means the "fastest" route (fewest jumps, biggest strides) isn't automatically the cheapest one — a cluster of small, careful jumps can easily beat one dramatic leap.

Example

Let's use `n = 4` with `cost = [_, 1, 2, 3, 4]` (step `0` has no cost since you start there for free).

One possible path: `0 → 1 → 2 → 4`

- Step `0 → 1`: jump distance 1, so cost = `cost[1] + 1² = 1 + 1 = 2`. Running total: **2**
- Step `1 → 2`: jump distance 1, cost = `cost[2] + 1² = 2 + 1 = 3`. Running total: **5**
- Step `2 → 4`: jump distance 2, cost = `cost[4] + 2² = 4 + 4 = 8`. Running total: **13**

Final answer: **13**.

Why does this beat the "obviously faster" plan of always jumping 3 steps? Because on this staircase, a single 3-step jump from `0` to `3` costs `cost[3] + 9 = 12`, and you still need another jump from there to finish — the squared penalty eats the time you saved. We'll quantify this properly in the Intuition section.

Intuition

The first temptation, for almost everyone, is to grab the farthest jump available every time. Fewer jumps feels like it should mean less total cost, since you're adding fewer `cost[j]` terms into the sum.

But look at what a 3-step jump actually costs *before* the landing step's own price even counts: `3² = 9`. Compare that to two separate 1-step jumps, which only cost `1² + 1² = 2` combined in penalties. The squared term grows much faster than the linear savings from skipping steps.

Concretely, on our example staircase, always jumping as far as possible (`0 → 3 → 4`) costs `(cost[3] + 9) + (cost[4] + 1) = 12 + 5 = 17`. Two balanced 2-step jumps (`0 → 2 → 4`) cost `(cost[2] + 4) + (cost[4] + 4) = 6 + 8 = 14`. And the mixed path we walked through above still wins at `13`. Greedy loses on every count.

Once you accept that greedy is out, the next observation unlocks the whole problem: whatever the cheapest cost to reach step `j` is, it can only have arrived via a jump from `j-1`, `j-2`, or `j-3` — there's no other way to land on `j`. That means the cheapest cost to reach `j` is simply:

```
best(j) = min(
    best(j-1) + cost[j] + 1²,   # jumped 1 step
    best(j-2) + cost[j] + 2²,   # jumped 2 steps
    best(j-3) + cost[j] + 3²,   # jumped 3 steps
)
```

restricted to whichever of those previous steps actually exist. This is the same recurrence shape as the unweighted "tribonacci" stair-counting problem — just with a price tag bolted onto every hop.

Brute Force Solution

Idea: Recursively try every legal jump from every reachable step, and take the minimum total cost across all complete paths to `n`.

Algorithm: 1. Define `solve(j)` = cheapest cost to reach step `j` from step `0`. 2. Base case: `solve(0) = 0`. 3. Recursive case: `solve(j) = min(solve(j-k) + cost[j] + k²)` for each valid `k` in `{1, 2, 3}` where `j-k >= 0`. 4. Return `solve(n)`.

Advantages: Trivial to reason about correctness; directly mirrors the problem statement.

Disadvantages: Without memoization, the same step gets recomputed from scratch along many overlapping paths, causing exponential blowup.

Complexity: Time is roughly `O(3^n)` in the naive recursive form (each call branches into up to 3 more). Space is `O(n)` for the recursion stack.

Optimal Solution

The recurrence above only ever looks at three neighbors behind the current step, and those sub-answers never change once computed. That's the textbook signal for bottom-up dynamic programming: build a table of size `n + 1`, fill it left to right, and never recompute anything.

Step	Looks back to	Formula
0	— (base case)	`dp[0] = 0`
1	0	`dp[1] = dp[0] + cost[1] + 1`
2	0, 1	`dp[2] = min(dp[1] + cost[2] + 1, dp[0] + cost[2] + 4)`
j ≥ 3	j-1, j-2, j-3	`dp[j] = min(dp[j-1]+cost[j]+1, dp[j-2]+cost[j]+4, dp[j-3]+cost[j]+9)`

Initialize `dp[0] = 0` and every other slot to infinity. Walking forward, for each step `j`, check the three possible launch points behind it, and only consider ones that actually exist (`j - k >= 0`). This naturally handles tiny staircases: when `n = 1`, a 2-step or 3-step jump would require a negative step index, so the existence check silently skips those options — no special-casing needed.

By the time the loop reaches `dp[n]`, that slot holds the answer.

Python Code

```
from typing import List


def min_cost_climbing_stairs(n: int, cost: List[int]) -> int:
    """Cheapest cost to climb from step 0 to step n, cost[0] unused."""
    dp = [float("inf")] * (n + 1)
    dp[0] = 0

    for j in range(1, n + 1):
        for k in (1, 2, 3):
            prev = j - k
            if prev >= 0:
                dp[j] = min(dp[j], dp[prev] + cost[j] + k * k)

    return dp[n]
```

Code Walkthrough

- `dp = [float("inf")] * (n + 1)` — one slot per step, all starting "unreachable" except the step we already know.
- `dp[0] = 0` — the starting step costs nothing because we begin there.
- `for j in range(1, n + 1)` — fill the table left to right; every step only needs steps already computed before it.
- `for k in (1, 2, 3)` — the only three jump lengths allowed by the problem.
- `if prev >= 0` — guards against jumping in from before step `0`; this is what makes tiny staircases (`n = 1` or `n = 2`) work without extra branches.
- `dp[j] = min(dp[j], dp[prev] + cost[j] + k * k)` — compare the current best for `j` against arriving via this particular jump, keep the cheaper one.
- `return dp[n]` — after the loop, the last slot holds the minimum cost for the whole staircase.

Dry Run

Using `n = 4`, `cost = [0, 1, 2, 3, 4]`:

- `dp = [0, inf, inf, inf, inf]`
- `j = 1`: only `k=1` valid (`prev = 0`). `dp[1] = dp[0] + cost[1] + 1 = 0 + 1 + 1 = 2`
- `j = 2`: `k=1` (`prev=1`): `dp[1] + cost[2] + 1 = 2 + 2 + 1 = 5`. `k=2` (`prev=0`): `dp[0] + cost[2] + 4 = 0 + 2 + 4 = 6`. Minimum is `5`. `dp[2] = 5`
- `j = 3`: `k=1` (`prev=2`): `5 + 3 + 1 = 9`. `k=2` (`prev=1`): `2 + 3 + 4 = 9`. `k=3` (`prev=0`): `0 + 3 + 9 = 12`. Minimum is `9`. `dp[3] = 9`

- `j = 4`: `k=1` (`prev=3`): `9 + 4 + 1 = 14`. `k=2` (`prev=2`): `5 + 4 + 4 = 13`. `k=3` (`prev=1`): `2 + 4 + 9 = 15`. Minimum is `13`. `dp[4] = 13`

Final answer: `dp[4] = 13`, matching the path `0 → 1 → 2 → 4` worked out earlier.

Complexity Analysis

Time: `O(n)`. Each of the `n` steps checks exactly 3 fixed neighbors, so total work scales linearly with the staircase length — no step is ever revisited.

Space: `O(n)` for the `dp` array. This can be trimmed to `O(1)` by keeping only the last three computed values instead of the full table, since nothing further back is ever needed.

Both bounds are tight: you must touch every step at least once to know its cost, and (in the array version) you need to retain enough history to look back three steps.

Alternative Solutions

A rolling-window variant keeps just three variables (`a`, `b`, `c` representing `dp[j-1]`, `dp[j-2]`, `dp[j-3]`) instead of a full array, dropping space to `O(1)` while keeping the same `O(n)` time. It's a worthwhile follow-up once the array version is solid, since interviewers often ask for the space optimization as a second pass.

A top-down memoized recursion (same recurrence, cached with a dictionary or `functools.lru_cache`) is equivalent in complexity to the bottom-up table and can feel more natural to write first, but it adds recursion-depth risk for large `n` (up to 100,000 per the constraints) unless you raise Python's recursion limit — the iterative version avoids that risk entirely.

Edge Cases

- **`n = 1`:** Only a 1-step jump is possible; 2- and 3-step jumps would require a negative index and get filtered out automatically by the `prev >= 0` check.
- **`n = 2`:** Both 1-step and 2-step paths are valid; the DP correctly compares `dp[1] + cost[2] + 1` against `dp[0] + cost[2] + 4`.
- **All costs equal:** Still worth testing — it isolates whether the jump-distance penalty alone (not varying step costs) drives the chosen path.
- **Strictly increasing or decreasing costs:** Checks that the algorithm doesn't assume any particular cost pattern.

- **Large `n` (close to 100,000):** Validates the `O(n)` solution actually runs fast; this is where the brute-force recursive version would time out.

Common Mistakes

1. **Always jumping the maximum distance**, assuming fewer jumps means lower cost — the squared penalty breaks that assumption.
2. **Forgetting the bounds check** on `j - k >= 0`, causing an `IndexError` or incorrectly reading `cost[-1]` for small `n`.
3. **Initializing `dp[0]` to something other than `0`**, which skews every downstream calculation.
4. **Confusing this problem with the unweighted "count the ways" variant** and summing counts instead of minimizing costs.
5. **Recomputing `k * k` incorrectly as `k * 2`** instead of squaring, silently producing wrong answers that are hard to spot by eye.
6. **Not initializing unreached slots to infinity**, which can make `min()` pick a stale default value like `0` instead of a genuinely unreachable state.

Interview Questions

- How would you reduce the space complexity from `O(n)` to `O(1)`?
- What changes if jumps of 4 or more steps were allowed?
- Can you prove, rather than just demonstrate with an example, that greedy fails here?
- How would you modify the solution to also return the actual sequence of jumps, not just the minimum cost?
- What if some steps had negative cost — does the same DP still work?

Similar Problems

- **LeetCode 70, Climbing Stairs** — the unweighted ancestor of this problem; same recurrence shape without the cost function.
- **LeetCode 746, Min Cost Climbing Stairs** — nearly identical idea with only 1- and 2-step jumps, a good warm-up before this one.
- **LeetCode 1137, N-th Tribonacci Number** — the pure numeric recurrence this problem's cost-free skeleton reduces to.
- **LeetCode 64, Minimum Path Sum** — different shape (grid instead of a line), but the same "cheapest cost depends only on nearby predecessors" DP principle.

Key Takeaways

Dynamic programming earns its keep exactly when a step's best answer depends only on a small, fixed set of earlier answers — build that answer forward once, instead of replaying every path from scratch. And greedy reasoning needs to be checked against the actual cost function, not just the apparent goal: fewer jumps is not the same thing as lower cost once a squared penalty enters the picture.

Watch the Video

For the full walkthrough — including a timed brute-force-versus-optimal run and two quiz rounds to lock in the intuition — watch the video here: {LINK}

About the Series

This breakdown is part of **Fun with Learning Technology**, a series that works through LeetCode's daily challenge and core interview topics with a focus on building real intuition before ever touching code — because the fastest way to remember a solution is to understand why it has to be that way.

Call To Action

If this breakdown helped the cost table click, leave a like on the video and subscribe for daily LeetCode walkthroughs. For the deeper written version of each problem, subscribe to the Substack newsletter. Drop a comment with your own approach or a problem you'd like covered next, and share this with anyone prepping for coding interviews.

Quick Review

What pattern signals a problem like Climbing Stairs II with variable jump sizes?

Linear DP / bottom-up tabulation — when each state depends on a small fixed set of prior states (prior steps), build answer from the base up.

Time complexity of the climbing-stairs-with-jump-costs DP?

O(n*k) where n is steps and k is max jump size, since each cell checks up to k prior entries.

Space complexity of the costs table approach?

O(n) — one array holding the best cost to reach each step; can often shrink to O(k) with a sliding window.

What's the trickiest bug when filling in the costs table?

Off-by-one on step 0 vs step 1 indexing, or forgetting that jumping far isn't always cheaper — must check all valid jump sizes, not just the max.

Why can jumping farther backfire in this problem?

A bigger jump may skip over a step with a much cheaper cumulative cost, so greedy max-jump isn't optimal — full DP comparison is required.

Interview follow-up: how would you reconstruct the actual path taken?

Store a parent/choice array alongside costs, then backtrack from the final step to step 0 following recorded choices.
